

PROJETO ECOPOWER MONITOR

DOCUMENTO DE ARQUITETURA E REQUISITOS (AEP — 1ª ENTREGA)

Instituição: Unicesumar — Curso de Engenharia de Software

Integrantes do Grupo: Victor Souza • Fernando Martinez • Samuel Hungaro

1. DESCOBERTA E CONCEPÇÃO (O PROBLEMA E A ODS)

O consumidor residencial só descobre quanto gastou de energia quando a fatura chega, no fim do mês. Nesse ponto, já não dá para corrigir nada: o desperdício de um aparelho esquecido ligado ou o pico de consumo de um mês mais quente viram um valor fechado na conta. Falta ao morador uma forma de acompanhar, ao longo do mês, para onde a energia está indo.

O **EcoPower Monitor** ataca esse problema. A aplicação registra o consumo de cada aparelho da casa, permite que o usuário defina uma meta mensal (em kWh ou em reais) e avisa quando o consumo se aproxima ou passa dessa meta — antes de a fatura fechar, quando ainda há o que fazer.

A proposta atende à **ODS 7 (Energia Limpa e Acessível)** e à **ODS 12 (Consumo e Produção Responsáveis)**. Dar visibilidade ao consumo incentiva o uso consciente da energia, reduz o desperdício dentro de casa e ajuda a família a se planejar financeiramente.

Partes interessadas e principais dores

O principal interessado é o morador responsável pela conta de luz. A pesquisa inicial com esse perfil apontou três dores concretas: a falta de informação durante o mês (só se sabe do gasto quando já passou), a dificuldade de identificar quais aparelhos pesam mais na conta, e a ausência de qualquer aviso antes de a fatura estourar o orçamento. Essas dores deram origem direta aos requisitos listados na próxima seção.

Uma decisão de projeto foi tomada logo no início: o sistema funciona de forma **local e autônoma**. A coleta e o registro das leituras acontecem em segundo plano, na própria máquina, sem depender de conexão constante com a internet ou de serviços externos. Isso simplifica o uso e mantém os dados sob controle do usuário.

2. LISTA DE REQUISITOS FUNCIONAIS (ESCOPO)

- **RF01 - Gestão de Dispositivos:** o sistema deve permitir cadastrar, editar e remover os aparelhos elétricos da residência, guardando de cada um o nome, o cômodo onde fica e a potência estimada em Watts.
- **RF02 - Registro de Leituras de Consumo:** o sistema deve registrar em segundo plano, e de forma automática, as medições de consumo (em kWh) de cada aparelho cadastrado, montando o histórico ao longo do tempo.
- **RF03 - Definição de Metas e Alertas:** o usuário deve poder definir uma meta mensal de consumo, em kWh ou em reais, e o sistema deve avisá-lo quando o consumo se aproximar do limite ou passar dele.
- **RF04 - Estimativa de Custos:** com base no consumo acumulado e na tarifa cadastrada pelo usuário, o sistema deve estimar quanto a conta de luz deve chegar em reais no fim do mês.
- **RF05 - Relatórios e Dicas de Economia:** o sistema deve gerar relatórios de consumo por aparelho e por período, e apontar quais equipamentos mais consomem, sugerindo onde o usuário pode economizar.

3. PLANEJAMENTO E CRONOGRAMA

O trabalho da 1ª entrega foi dividido entre os três integrantes, com uma frente por semana. A tabela abaixo mostra o período, a atividade e o responsável por cada etapa. Ela também consta no arquivo README do repositório, conforme exigido pela AEP.

Semana	Período	Atividade / Entrega	Responsável
Semana 1	04/08 a 10/08	Levantamento do problema, definição do escopo e requisitos (Engenharia de Requisitos).	Victor Souza
Semana 2	11/08 a 15/08	Modelagem Orientada a Objetos e Diagrama de Classes UML.	Fernando Martinez
Semana 3	17/08 a 21/08	Modelagem do Banco de Dados Relacional (DER / PostgreSQL) e script SQL.	Samuel Hungaro
Semana 4	23/08 a 28/08	Configuração do ambiente (JDK, VS Code, PostgreSQL local) e organização do repositório.	Victor Souza
Semana 5	01/09 a 11/09	Revisão final, consolidação da documentação e preparação da 1ª Entrega da AEP.	Victor Souza

4. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

As escolhas abaixo foram guiadas por dois fatores: o domínio que a equipe já tem sobre as ferramentas e a necessidade de deixar visível, no código, cada uma das disciplinas do semestre. Optamos por não usar frameworks pesados para não esconder esses conceitos atrás de automações.

Linguagem — Java puro. Escolhemos Java pela tipagem forte e por ser a linguagem em que o grupo tem mais base. Trabalhar sem framework nos obriga a implementar a orientação a objetos na mão, o que atende ao que a AEP cobra: **encapsulamento** nos atributos das classes de modelo, **herança** entre os tipos de usuário e **polimorfismo** na geração dos relatórios. Um framework como Spring resolveria parte disso automaticamente, mas aí esses pilares não apareceriam no nosso código.

Banco de dados — PostgreSQL local, via JDBC. Usamos PostgreSQL instalado na máquina, acessado por **JDBC** com o SQL escrito pela própria equipe. As consultas ficam concentradas em classes **DAO**, uma por tabela, o que separa o acesso ao banco do resto do programa. Preferimos escrever o SQL a usar um ORM justamente para mostrar domínio de consultas, junções e agregações — que é o que a disciplina de Banco de Dados avalia. A gestão visual do banco é feita no **DBeaver**, e o driver de conexão é baixado pelo **Maven**.

Coleta em segundo plano — threads do Java. A parte de Sistemas Operacionais aparece na coleta das leituras. Cada aparelho monitorado gera suas leituras em uma **thread** própria, rodando em paralelo. Como várias threads gravam no banco ao mesmo tempo, controlamos esse acesso com **exclusão mútua** (**synchronized**), evitando condição de corrida. É um caso concreto de concorrência e sincronização, e não um exemplo artificial encaixado só para cumprir a disciplina.

Ambiente e execução. O projeto roda em qualquer sistema com o JDK instalado, já que o Java é multiplataforma. As instruções de instalação das dependências (JDK, PostgreSQL e driver JDBC) e de execução ficam documentadas no README do repositório, conforme pede a orientação da disciplina de SO.

5. AMBIENTE DE CONTROLE DE VERSÃO

O código e a documentação do projeto ficam versionados no GitHub. O repositório é público e está organizado nas pastas **/src** (código-fonte), **/docs** (documentação) e **/database** (script SQL de criação das tabelas). O arquivo README reproduz a lista de requisitos e o cronograma apresentados neste documento.

Repositório: <https://github.com/VictorsouzaV/ecopower.git>